

Chaos Backend — шпаргалка к собесу (мидл)

Цель: уметь за час собеседа уверенно рассказать про бекенд и увести разговор в сильные кейсы.

Проговаривай вслух. Не зубрить дословно — держать структуру и причины решений.

1. Elevator pitch (60–90 сек)

Chaos — production-oriented multi-device E2EE мессенджер. Я делал бекенд на **Java 17 / Spring Boot 3.5**, PostgreSQL + Flyway, Redis, Kafka-compatible transport (Redpanda), realtime через STOMP/SockJS.

Четыре инженерные цели:

- 1. **Plaintext остаётся на устройствах** — сервер хранит ciphertext и envelopes, не видит текст сообщений.
- 2. **Доставка переживает сбои** — transactional outbox → Kafka → durable device event store + cursor sync.
- 3. **Auth учитывает кражу токена** — refresh rotation, revoke family при reuse, отдельный one-time token на регистрацию устройства.
- 4. **Систему можно эксплуатировать** — Compose/K8s, метрики, runbooks.

Честно: это не Signal Protocol и крипто-аудит не делали. Я проектировал сервер вокруг trust boundary и durable delivery, а не вокруг «мы как Signal».

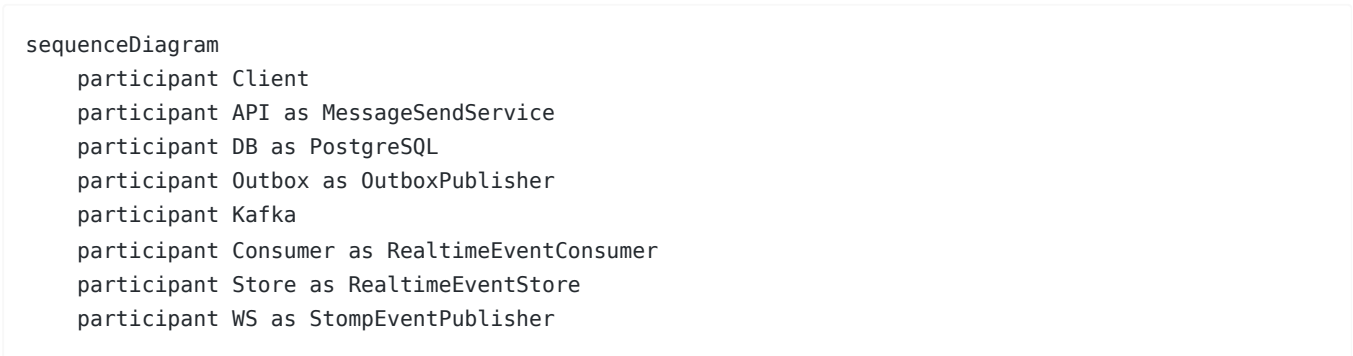
Крючок: «Самое интересное на беке — не CRUD чатов, а гарантированная доставка в multi-replica + WebSocket как оптимизация».

2. Стек одной строкой

Слой	Технологии
API	Spring MVC, springdoc-openapi
Security	Spring Security, JWT (access), Redis refresh families
Data	JPA/Hibernate (open-in-view=false, ddl-auto=validate), Flyway
Cache/coord	Redis: presence, unread, rate limits, tokens
Messaging	Outbox + Spring Kafka (Redpanda)
Realtime	STOMP + SockJS /ws, SimpleBroker
Ops	Actuator/Prometheus, Docker Compose, K8s

Пакеты по доменам: `auth`, `user`, `chat`, `message`, `crypto`, `outbox`, `realtime`, `push`, `attachment`, `infra`. Слои проверяются ArchUnit.

3. Архитектура: путь сообщения (главная схема)



```
Client->>API: encrypted envelopes + clientMessageId
API->>DB: message + envelopes + outbox в одной TX
Note over API: afterCommit: unread/push; local STOMP только если Kafka OFF
Outbox->>Kafka: DomainEvent sync send
Kafka->>Consumer: at-least-once delivery
Consumer->>Store: append device event ON CONFLICT DO NOTHING
Consumer->>WS: notify если сессия на этом поде
Client->>API: GET /api/realtime/sync?after=cursor
```

Что говорить по шагам

1. Клиент шифрует **отдельный envelope на каждое устройство** получателя (и на свои другие девайсы).
2. `MessageSendService.sendEncryptedMessageV2` в одной TX:
 - идемпотентность по `(senderId, senderDeviceId, clientMessageId)` ;
 - `content = "[encrypted]"` ;
 - `persist envelopes`;
 - `write` в `outbox ( MessageOutboxService )`.
3. `TransactionUtils.afterCommit` :
 - `unread + Web Push` для оффлайн;
 - локальный STOMP **только если** `chaos.kafka.enabled=false` .
4. Если Kafka ON: `OutboxPublisher (scheduled poll)` → Kafka → `RealtimeEventConsumer` :
 - пишет в `realtime_device_events` ;
 - шлёт STOMP, если устройство подключено к **этому** экземпляру.
5. Клиент при реконнекте тянет `GET /api/realtime/sync?after=` — это source of truth. WS — ускоритель.

Ключевые классы

Класс	Роль
<code>MessageSendService</code>	send + idempotency + afterCommit
<code>OutboxService / OutboxPublisher</code>	write/claim/publish
<code>RealtimeEventConsumer</code>	Kafka → durable + STOMP
<code>RealtimeEventStore</code>	sequence + cursor sync
<code>StompEventPublisher</code>	publish только при local session
<code>TransactionUtils</code>	не держать DB connection на fan-out
<code>WebSocketAuthChannelInterceptor</code>	CONNECT/SUBSCRIBE ACL

4. Топ кейсов (разбирать на собеседе)

Формат каждого: **проблема** → **решение** → **шаги** → **trade-off** → **вопрос интервьюера**.

Кейс 1. Transactional outbox

Проблема. Если писать в БД и в Kafka в одном HTTP-запросе:

- Kafka ок, DB rollback → фантомное событие;
- DB ок, Kafka упала → потеря события.

Решение. В той же TX, что бизнес-данные, пишем строку в `outbox_events` . Отдельный поллер публикует в Kafka.

Шаги в коде

1. `OutboxService.write` — `@Transactional(propagation = MANDATORY)` : нельзя писать outbox вне чужой TX.
2. Статусы: `PENDING` → `PROCESSING` → `PUBLISHED` | `FAILED` → `DEAD` (max 10 attempts, backoff).
3. Claim: `FOR UPDATE SKIP LOCKED` — несколько реплик не воруют одни и те же события.
4. `OutboxPublisher` каждую ~1s: release stale locks (120s) → claim batch → `kafkaTemplate.send(...).get(timeout)` → `markPublished` / `markFailure`.
5. Partition key для `message/chat/request` = `aggregateId` → порядок событий одного чата в партиции.

Trade-off. Latency выше, чем «сразу в Kafka». Зато атомарность с бизнес-TX и multi-instance safe publish.

Спросят: «Почему не CDC/Debezium?» — можно, но outbox проще контролировать payload/версию события и не тащить raw row changes. У нас явный domain event.

Сaveat честно: outbox-строки пишутся даже при `kafka.enabled=false` ; publisher просто не активен. Локально fan-out идёт напрямую в STOMP.

Кейс 2. At-least-once end-to-end

Проблема. Exactly-once по всей цепочке (DB → Kafka → DB → WS → client) практически недостижим без тяжёлой семантики. Нужна доставка без потери при ретраях.

Решение. Сознательный **at-least-once** + идемпотентность на каждом слое.

Слой	Как
Producer	<code>acks=all</code> , <code>enable.idempotence=true</code> , <code>retries</code> , <code>sync send</code>
Consumer	<code>enable.auto.commit=false</code> , <code>retry</code> + DLQ (<code>DefaultErrorHandler</code> + <code>DeadLetterPublishingRecoverer</code>)
Durable store	<code>ON CONFLICT (device_id, event_id, destination) DO NOTHING</code>
In-memory dedupe	<code>processedEvents</code> по <code>eventId</code> — только после успеха (иначе отправишь ретраи)
Client	dedupe по <code>eventId</code> , <code>cursor</code> по <code>sequence</code>

Почему не exactly-once. Kafka EOS помогает `producer`→`broker`→`consumer` TX, но у нас ещё Postgres durable store, `SimpleBroker` и клиент. Дешевле сделать идемпотентный consumer + клиентский dedupe.

Фраза: «Мы гарантируем “как минимум один раз”, а “ровно один эффект” получаем идемпотентностью».

Спросят: «Что если consumer упал после записи в store, но до ack?» — при ретрае `ON CONFLICT DO NOTHING` , STOMP может уйти повторно, клиент отфильтрует по `eventId` .

Кейс 3. Unique Kafka consumer group на каждый pod

Проблема. STOMP `SimpleBroker` хранит сессии **в памяти пода**. Если все реплики в одной consumer group — событие получит только один pod. Пользователи на других подах не увидят realtime.

Решение. Group id уникален на инстанс:

```
chaos.kafka.realtime.group-id=chaos-realtime-${random.uuid}
```

В k8s обычно привязка к `HOSTNAME` . Каждый backend читает **все** события и нотифицирует только свои локальные WS-сессии.

Trade-off. Это broadcast-consume: нагрузка на consumer растёт линейно с числом реплик. Для текущего масштаба ок. Эволюция — внешний STOMP broker (RabbitMQ/Redis relay) и тогда уже shared consumer group + publish в relay.

Спросят: «Это же не классический scaling consumers?» — да, здесь Kafka используется как fan-out bus между репликами приложения, не как шардированная обработка work-queue.

Кейс 4. Dual-mode fan-out (kafkaEnabled)

Проблема. Локально/в тестах не всегда нужен брокер, в кластере — нужен.

Решение. Флаг `chaos.kafka.enabled` :

- **false:** `MessageFanoutService` шлёт STOMP напрямую после commit;
- **true:** локальный STOMP для domain events пропускается; путь только outbox → Kafka → consumer.

Beans `KafkaConfig` , `OutboxPublisher` , `RealtimeEventConsumer` — `@ConditionalOnProperty(...=true)` .

Спросят: «Не разъедется ли семантика?» — да, риск. Поэтому correctness всё равно строится на durable store + sync API в kafka-режиме; локальный режим — dev convenience.

Кейс 5. WebSocket = оптимизация, sync = correctness

Проблема. WS рвётся. Нельзя строить доставку только на live-канале.

Решение.

1. Endpoint `/ws` + SockJS, prefixes `/topic` , `/queue` , app `/app` .
2. CONNECT: `Authorization: Bearer` + `X-Device-Id` , устройство active, регистрация в `WebSocketSessionRegistry` .
3. SUBSCRIBE ACL:
 - `/topic/devices/{deviceId}/...` — только своё устройство сессии;
 - `/topic/users/{username}/chats|requests` — только свой username;
 - `/topic/chats/{id}/typing` — participant;
 - `/topic/user/status` — любой auth user.
4. `StompEventPublisher` не шлёт, если сессии устройства нет локально.
5. Recovery: `RealtimeEventStore.readAfter` → GET `/api/realtime/sync` , retention ~7 дней, hourly cleanup.

Фраза: «WebSocket ускоряет доставку online; correctness даёт cursor sync».

Кейс 6. afterCommit для fan-out

Проблема. STOMP внутри открытой TX держит JDBC connection → под нагрузкой pool exhaustion.

Решение. `TransactionUtils.afterCommit` регистрирует callback; вне TX выполняет сразу.

Используется в send/edit/delete/receipts/reactions/self-destruct/chat updates.

Спросят: «А если afterCommit упадёт?» — сообщение уже в БД (+ outbox). При Kafka ON доставка догонит через publisher/consumer. При Kafka OFF — риск, поэтому в проде Kafka ON.

Кейс 7. Auth: refresh families + device registration

Access JWT — короткий, stateless, HS256, issuer/audience.

Refresh (`RefreshTokenService` , Redis):

- храним SHA-256 digest, не raw token;
- каждый refresh: consume old (`getAndDelete`) → пометить used → выдать новый той же family;
- если пришёл уже used token → **revoke всей family** (признак кражи/replay).

Device registration token — короткоживущий one-time в Redis, отдельно от JWT, чтобы enroll crypto-device не раздуть права access token.

Плюс Redis sliding-window rate limits на credentials/SMS.

Фраза: «Модель threat: access украдут — коротко живёт; refresh украдут и реюзнут — убиваем всю семью».

Кейс 8. E2EE trust boundary на сервере

Сервер **может** знать: аккаунт, device ids, публичные keys/prekeys, состав чатов, ciphertext envelopes, delivery/timing metadata, push subscription metadata.

Сервер **не должен** получать: plaintext, private keys, ratchet keys, backup passphrase.

Практические следствия бека:

- `Message.content = "[encrypted]"` ;
- fan-out по device envelopes;
- OPTK reservation через `FOR UPDATE SKIP LOCKED` при конкурентных bundle fetch;
- Safety Number / KEY_CHANGED — клиентская trust UX, сервер только доставляет материал.

Не ври, что «сервер ничего не знает» — метаданные видит. Это нормальный честный ответ на мидле.

Кейс 9. Идемпотентность send

Клиент шлёт `clientMessageId` . Unique constraint + pre-check + catch `DataIntegrityViolationException` на race → вернуть тот же результат.

Зачем: ретраи HTTP при таймауте не создают дубликаты сообщений.

Кейс 10. Коротко, если спросят

Тема	Суть
Presence	Redis session set; ONLINE/OFFLINE на transition; heartbeat ~25s
Self-destruct	scheduler каждые 30s, soft delete + afterCommit fan-out
Unread	Redis counters, инкремент afterCommit
Attachments	ciphertext на FS (atomic write); S3 SDK в зависимостях, путь эволюции
Observability	chaos_outbox_*, chaos_kafka_*, chaos_ws_*, runbooks
ArchUnit	controllers → repositories; services → controllers
Не делали	saga/CQRS split, Resilience4j CB, external STOMP relay

5. Устный скрипт «расскажи о проекте» (5–7 мин)

Можно почти дословно, но своими словами:

Я делал бекенд для Chaos — multi-device E2EE мессенджера. Стек: Java 17, Spring Boot, Postgres, Redis, Kafka-compatible брокер, STOMP WebSocket.

Главный use-case — отправка зашифрованного сообщения. Клиент присылает envelopes на каждое устройство. В одной транзакции я сохраняю message, envelopes и outbox-событие. В content лежит плейсхолдер `[encrypted]` — plaintext сервер не видит.

Публикацию в Kafka не делаю внутри HTTP-транзакции: transactional outbox + поллер с `SKIP LOCKED` , чтобы несколько реплик безопасно публиковали. Producer с `acks=all` и idempotence.

Consumer пишет durable device events и пытается пнуть STOMP. Важно: у нас SimpleBroker, сессии локальные, поэтому у каждой реплики свой Kafka consumer group — иначе realtime увидит только один под. При этом WebSocket для меня оптимизация: если клиент был офлайн или сменил под, он догоняет историю через `/api/realtime/sync` по cursor. Семантика доставки — at-least-once, дубли режим eventId/sequence.

По auth: короткий JWT, refresh в Redis с ротацией и revoke family при reuse. Регистрация crypto-device — отдельный one-time token.

Если развивать дальше — вынести STOMP в external broker, чтобы не broadcast-consume Kafka на каждый под, и вынести attachments в S3.

Куда уводить вопросы

- «Как гарантируете доставку?» → outbox + at-least-once + sync
- «Как масштабируете WS?» → unique group / trade-off / relay
- «Что с безопасностью?» → trust boundary + refresh families
- «Где гонки?» → clientId, SKIP LOCKED outbox/OPTK

6. Q&A (проговаривай вслух)

Архитектура

Q: Это монолит или микросервисы?

A: Модульный монолит по пакетам. Для текущего размера проще транзакции и outbox в одном процессе. Границы доменов уже видны — можно резать позже.

Q: Почему не писать в Kafka сразу из сервиса?

A: Dual-write. Outbox даёт атомарность с бизнес-данными и ретраи без потери.

Q: Где граница консистентности?

A: Message+envelopes+outbox — strong в одной TX. Realtime/WS — eventually, догоняется sync.

Q: Open-in-view выключен — зачем?

A: Не держать сессию Hibernate на весь HTTP-рендер/сериализацию; явные fetch в сервисе, предсказуемые транзакции.

Kafka / outbox

Q: At-least-once vs at-most-once vs exactly-once?

A: At-most-once — можно потерять. Exactly-once end-to-end дорого/хрупко. At-least-once + idempotency — наш выбор для мессенджера.

Q: Что делает DLQ?

A: После exponential backoff (DefaultExceptionHandler) битое событие уходит в `chaos.dead-letter.events`, не блокирует партицию бесконечно.

Q: Зачем sync send в publisher (.get())?

A: Чтобы markPublished только после подтверждённой отправки. Цена — throughput поллера; для outbox batch это осознанно.

Q: Что если outbox завис в PROCESSING?

A: stale lock release через 120s, событие снова PENDING.

Q: Порядок сообщений?

A: Partition key = chat/message aggregateId → порядок в пределах партиции. Глобального порядка между чатами нет и не нужен.

Q: Почему in-memory dedupe недостаточно?

A: Он ускоряет. Настоящая идемпотентность — unique в `realtime_device_events` и клиентский eventId.

WebSocket

Q: SockJS зачем?

A: Fallback-транспорты за прокси/файрволами; STOMP поверх.

Q: Как не подписаться на чужой device topic?

A: Interceptor на SUBSCRIBE сверяет path deviceId с deviceId сессии + active device в БД.

Q: Typing и presence durable?

A: Нет, ephemeral. Durable — сообщения/статусы/chat list events.

Q: Что если пользователь online на двух девайсах?

A: Fan-out по deviceId; каждое устройство получает свой envelope и свой durable log.

Транзакции / конкуренция**Q: FOR UPDATE SKIP LOCKED — зачем?**

A: Воркер берёт свободные строки и не ждёт заложенные. Outbox и OPTK.

Q: Race на повторный send?

A: Unique + catch integrity violation → idempotent response.

Q: Почему MANDATORY на outbox write?

A: Чтобы нельзя было «забыть» обернуть в бизнес-TX и получить outbox без message или наоборот по смыслу вызова.

Security**Q: CSRF выключен?**

A: Stateless Bearer API, не cookie-session форма. Для WS отдельная JWT-проверка на CONNECT.

Q: Что будет при утечке refresh?

A: Пока не реюзнули — как обычная сессия до TTL. При reuse старого после ротации — revoke family, остальные refresh этой семьи мертвы.

Q: Сервер видит метаданные — это проблема?

A: Да, классический metadata leak E2EE. Честно говорим; защита содержимого ≠ скрывание графа общения.

Scaling / эволюция / слабости**Q: Узкое место сейчас?**

A: Broadcast Kafka consume на каждый pod из-за SimpleBroker; outbox poller latency; attachments на локальном FS.

Q: Как бы масштабировал WS?

A: External broker / Redis relay → одна consumer group обрабатывает события и публикует в relay; поды только держат WS.

Q: Почему Redis для refresh, а не БД?

A: TTL, быстрый getAndDelete, меньше нагрузки на Postgres для hot path auth. Минус — зависимость от Redis availability.

Q: Что бы сделал иначе?

A: Раньше заложить external broker; явный контракт dual-mode (dev vs prod) через один delivery pipeline; S3 для ciphertext blobs.

Q: Тесты?

A: Unit/integration, Testcontainers (Postgres + Kafka), ArchUnit на слои. На себе можно упомянуть тесты consumer dedupe «только после успеха».

7. Чеклист за день до собеса

- ☐ Нарисовать на бумаге путь сообщения (TX → outbox → Kafka → store → WS → sync).
- ☐ Объяснить at-least-once vs exactly-once на своём примере за 60 секунд.
- ☐ Объяснить, зачем unique consumer group, и назвать trade-off.
- ☐ Назвать 2 слабости и 1 следующий шаг эволюции.
- ☐ Проговорить refresh reuse → revoke family.
- ☐ Проговорить, что сервер видит/не видит в E2EE.
- ☐ Вспомнить имена классов из таблицы выше (не файлы наизусть — роли).

Мини-тренировка (15 минут)

1. Пич 90 сек.
 2. Outbox 2 мин.
 3. At-least-once 2 мин.
 4. WS+sync 2 мин.
 5. Auth families 1 мин.
 6. «Что улучшить» 1 мин.
-

8. Быстрые якоря (если завис)

Вопрос из зала	Якорь
Доставка	outbox + at-least-once + sync cursor
Реплики	unique group из-за SimpleBroker
Дубли	eventId + unique constraint + client dedupe
Тормоза pool	afterCommit
Безопасность сообщений	ciphertext only / metadata visible
Безопасность сессии	refresh rotate + revoke family

Удачи. Веди разговор в эти кейсы сам — на мидле это выглядит сильнее, чем ждать «расскажите про Spring аннотации».